build a highly dynamic web app, you might want to consider using minimal APIs or web API controllers with a client-side framework, or consider using Blazor instead.

That concludes our first look at rendering HTML using the Razor templating engine. In the next chapter you'll learn about Tag Helpers and how to use them to build HTML forms, a staple of modern web applications. Tag Helpers are one of the biggest improvements to Razor in ASP.NET Core over legacy ASP.NET, so getting to grips with them will make editing your views an overall more pleasant experience!

Summary

- Razor is a templating language that allows you to generate dynamic HTML using a mixture of HTML and C#. This provides the power of C# without your having to build up an HTML response manually using strings.
- Razor Pages can pass strongly typed data to a Razor view by setting public properties on the `PageModel`. To access the properties on the view model, the view should declare the model type using the `@model` directive.
- Page handlers can pass key-value pairs to the view using the `ViewData` dictionary. This is useful for implicitly passing shared data to layouts and partial views.
- Razor expressions render C# values to the HTML output using `@` or `@()`. You don't need to include a semicolon after the statement when using Razor expressions.
- Razor code blocks, defined using `@{}`, execute C# without outputting HTML. The C# in Razor code blocks must be complete statements, so it must include semicolons.